Review Problem 13

- Write assembly to compute X1 = X0*5 without using a multiply or divide instruction.

```
ADD x1, x0, x0
ADD x1, x1, x0
ADD x1, x1, x0
ADD x1, x1, x0
```

```
LSL x1, x0, #2
ADD x1, x1, x0
```

```
ADD x1, x0, x0
ADD x1, x1, x1
ADD x1, x1, x0
```

Computer Performance

Primary goal: execution time (time from program start to program completion)

$$Performance = \frac{1}{ExecutionTime}$$

To compare machines, we say "X is n times faster than Y"

$$n = \frac{Performance_x}{Performance_y} = \frac{ExecutionTime_y}{ExecutionTime_x}$$

Example: Machine *Orange* and *Grape* run a program

Orange takes 5 seconds, Grape takes 10 seconds

$$\frac{Perf_{orange}}{Perf_{grape}} = \frac{Exec_{grape}}{Exec_{orange}} = \frac{10}{5} = 2$$

Orange is 2 times faster than Grape

Execution Time

Elapsed Time

counts everything *(disk and memory accesses, I/O , etc.)*

a useful number, but often not good for comparison purposes

CPU time

doesn't count I/O or time spent running other programs

can be broken up into system time, and user time

Example: Unix "time" command

```
linux15.ee.washington.edu> time javac CircuitViewer.java
3.370u 0.570s 0:12.44 31.6%
```

user system elapsed u+s/elapsed

Our focus: user CPU time

time spent executing the lines of code that are "in" our program

1 GHz = 1ns
Rate = period

G = 10^9 cycles
ns = 10^{-9} second

CPU Time

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} * \text{Clock period}$$

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} * \frac{1}{\text{Clock rate}}$$

Application example:

A program takes 10 seconds on computer *Orange*, with a 400MHz clock. Our design team is developing a machine *Grape* with a much higher clock rate, but it will require 1.2 times as many clock cycles. If we want to be able to run the program in 6 second, how fast must the clock rate be?

$Exec_{orange} = cycles_{orange} \times \frac{1}{Rate_{orange}}$

$10 = cycles_{orange} \times \frac{1}{0.4\times10^9}$

$cycles_{orange} = 0.4\times10^9 \times 10$

$= 4\times10^9$ cycles

$Exec_{grape} = cycles_{grape} \times \frac{1}{Rate_{grape}}$

$6 = (1.2\times4\times10^9) \times \frac{1}{Rate_{grape}}$

$Rate_{grape} = \frac{1.2\times4\times10^9}{6}$

$= 0.2\times4\times10^9$

$= 0.8\times10^9$

800 MHz

CPI

How do the # of instructions in a program relate to the execution time?

$$\text{CPU clock cycles for a program} = \text{Instructions for a program} * \text{Average Clock Cycles per Instruction (CPI)}$$

$$\text{CPU execution time for a program} = \text{Instructions for a program} * \text{CPI} * \frac{1}{\text{Clock rate}}$$

$$* \text{ Period}$$

CPI Example

Suppose we have two implementations of the same instruction set (ISA).

For some program

Machine A has a clock cycle time of 10 ns. and a CPI of 2.0
Machine B has a clock cycle time of 20 ns. and a CPI of 1.2

What machine is faster for this program, and by how much?

$$\frac{Perf_A}{Perf_B} = \frac{Exec_B}{Exec_A} = \frac{Instr_B \times CPI_B \times Period_B}{Instr_A \times CPI_A \times Period_A} = \frac{1.2 \times 20}{2.0 \times 10} = \frac{24}{20} = 1.2\times$$

Computing CPI

Different types of instructions can take very different amounts of cycles

Memory accesses, integer math, floating point, control flow

$$CPI = \sum_{types} \left(Cycles_{type} * Frequency_{type} \right)$$

Instruction Type	Type Cycles		Type Frequency		Cycles * Freq
ALU	1	×	50%	=	0.5
Load	5	×	20%	=	1.0
Store	3	×	10%	=	0.3
Branch	2	×	20%	=	0.4
			CPI:		2.2

$\frac{Perf_{new}}{Perf_{old}} = \frac{Exec_{old}}{Exec_{new}} = \frac{Inst_{old} \times CPI_{old} \times Period_{old}}{Inst_{new} \times CPI_{new} \times Period_{new}} = \frac{CPI_{old}}{CPI_{new}} = \frac{2.2}{CPI_{new}}$

CPI & Processor Tradeoffs

Instruction Type	Type Cycles	Type Frequency
ALU	1	50%
Load	5	20%
Store	3	10%
Branch	2	20%

2.2

How much faster would the machine be if:

1. A data cache reduced the average load time to 2 cycles?

$$\frac{2.2}{(1\times.5 + 2\times.2 + 3\times.1 + 2\times.2)} = \frac{2.2}{0.5+0.4+0.3+0.4} = \frac{2.2}{1.6} = 1.375\times$$

2. Branch prediction shaved a cycle off the branch time?

$$\frac{2.2}{(1\times.5 + 5\times.2 + 3\times.1 + 1\times.2)} = \frac{2.2}{0.5+1.0+0.3+0.2} = \frac{2.2}{2.0} = 1.1\times$$

3. Two ALU instructions could be executed at once?

$$\frac{2.2}{(1/2\times.5 + 5\times.2 + 3\times.1 + 2\times.2} = \frac{2.2}{0.25+1.0+0.3+0.4} = \frac{2.2}{1.95} = 1.13\times$$